BRNO UNIVERSITY OF TECHNOLOGY
VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

FACULTY OF INFORMATION TECHNOLOGY
FAKULTA INFORMAČNÍCH TECHNOLOGIÍ

DEPARTMENT OF INTELLIGENT SYSTEMS
ÚSTAV INTELIGENTNÍCH SYSTÉMŮ

AN ENVIRONMENT FOR THE VISUAL DESIGN, MONITORING, AND MANAGEMENT OF DISTRIBUTED CONTROL SYSTEMS AND IOT NETWORKS

PROSTŘEDÍ PRO VIZUÁLNÍ NÁVRH, MONITOROVÁNÍ A SPRÁVU DISTRIBUOVANÝCH ŘÍDICÍCH SYSTÉMŮ A IOT SÍTÍ

BACHELOR'S THESIS
BAKALÁŘSKÁ PRÁCE

AUTHOR **JAKUB KAPITULCIN**
AUTOR PRÁCE

SUPERVISOR **doc. Ing. VLADIMÍR JANOUŠEK, Ph.D.**
VEDOUCÍ PRÁCE

BRNO 2026

Abstract

This thesis presents Aetherium Automata, a platform for the visual design, deployment, and monitoring of distributed IoT control systems modeled as Extended Finite State Machines (EFSMs). The platform consists of a portable C++17 runtime engine, a real-time Elixir/Phoenix gateway, and an Electron-based visual IDE. The EFSM model supports five transition types—classic guarded, timed, event-driven, probabilistic, and immediate—expressed in a YAML-based domain-specific language. The engine executes Lua scripts for guards and state actions and targets heterogeneous hardware including desktop systems, ESP32, and Raspberry Pi Pico. The IDE provides live state monitoring, variable inspection, and time-travel debugging via execution trace replay. Structural analysis using Petri net formalism enables deadlock detection and bottleneck identification across multi-automata networks. The system is demonstrated through a catalog of 18 showcase automata covering a range of real-world IoT scenarios.

Abstrakt

Tato práce představuje platformu Aetherium Automata pro vizuální návrh, nasazení a monitorování distribuovaných řídicích systémů IoT modelovaných jako rozšířené konečné automaty. Platforma se skládá z přenositelného runtime jádra v jazyce C++17, brány v Elixiru a Phoenix, a vizuálního vývojového prostředí postaveného na technologii Electron. Model automatu podporuje pět typů přechodů a doménově specifický jazyk v YAML. Jádro je spustitelné na stolních počítačích i na vestavěných platformách jako ESP32 a Raspberry Pi Pico. IDE umožňuje živé monitorování, inspekci proměnných a ladění s přetáčením času. Strukturální analýza pomocí Petriho sítí umožňuje detekci deadlocků a identifikaci úzkých hrdel v sítích automatů.

Keywords

extended finite state machine, Petri net, IoT, distributed control systems, state machine, Lua, embedded systems, visual IDE, fault injection, time-travel debugging

Klíčová slova

rozšířený konečný automat, Petriho síť, IoT, distribuované řídicí systémy, stavový stroj, Lua, vestavěné systémy, vizuální IDE, vstřikování chyb, ladění s přetáčením času

Reference

KAPITULCIN, Jakub. *An environment for the visual design, monitoring, and management of distributed control systems and IoT networks.* Brno, 2026. Bachelor's thesis. Brno University of Technology, Faculty of Information Technology. Supervisor doc. Ing. Vladimír Janoušek, Ph.D.

An environment for the visual design, monitoring, and management of distributed control systems and IoT networks

Declaration

I hereby declare that this Bachelor's thesis was prepared as an original work by the author under the supervision of doc. Ing. Vladimír Janoušek, Ph.D. I have listed all the literary sources, publications, and other sources which were used during the preparation of this thesis. I used Claude Code (an AI programming assistant by Anthropic) for code assistance and documentation generation during the development of the implemented system.

.....
Jakub Kapitulcin
May 11, 2026

Acknowledgements

I would like to thank doc. Ing. Vladimír Janoušek, Ph.D. for his guidance throughout the work on this thesis.

Contents

List of Figures

Chapter 1

Introduction

The Internet of Things (IoT) connects increasingly large numbers of heterogeneous devices that must cooperate to execute control logic in real time. Such systems are inherently distributed and stateful: a smart thermostat cycles through heating, idle, and cooling modes; a production-line robot arm progresses through pick, move, and place phases; a sensor watchdog transitions between monitoring, alarming, and recovery states. The heterogeneity of devices, communication protocols, and data formats in large IoT deployments is a well-documented integration challenge [12]. Despite the pervasiveness of state-driven behavior, most existing IoT toolchains treat control logic as ad-hoc scripts embedded in event callbacks or workflow DAGs, making the underlying state structure implicit and difficult to reason about.

The consequence is a practical problem: when a system deployed across dozens of devices misbehaves, developers must reconstruct the global state from scattered log messages, often without any ability to replay or visually inspect the sequence of events that led to the fault. Testing distributed IoT systems reliably requires simulating faults and reproducing timing-sensitive failure scenarios [1], a capability that most existing IoT platforms do not provide natively.

This thesis addresses that problem by presenting *Aetherium Automata*, a complete toolchain for the visual design, deployment, and monitoring of distributed IoT control systems modeled as *Extended Finite State Machines* (EFSMs). The system treats automata as first-class citizens: every device behavior is expressed as a named state machine with typed variables, Lua-scripted guards and actions, and a rich set of transition types. The toolchain includes:

- a portable C++17 runtime engine that executes EFSM definitions on targets ranging from desktop Linux to ESP32 and Raspberry Pi Pico,
- a YAML-based domain-specific language (DSL) for authoring automata,
- a real-time Elixir/Phoenix gateway that bridges IDE and device network,
- an Electron-based visual IDE for design, live monitoring, fault injection, and time-travel debugging,
- structural analysis using Petri net formalism for deadlock and bottleneck detection across multi-automata networks.

1.1 Goals

The primary goals of this thesis are:

1. Design and implement an EFSM model expressive enough to cover the control logic of real-world IoT devices, including time-driven, event-driven, and probabilistic behavior.
2. Implement a portable C++ runtime engine capable of executing the model on both desktop and constrained embedded platforms.
3. Design a YAML domain-specific language for authoring automata definitions.
4. Implement a real-time gateway and IDE that allow an operator to deploy, monitor, and debug automata across a network of heterogeneous devices.
5. Validate the system with a catalog of representative IoT showcase automata.

1.2 Thesis Structure

Chapter 2 introduces the theoretical background: Finite State Machines, Extended Finite State Machines, and Petri nets. Chapter 3 discusses related tools and analyses requirements. Chapter 4 describes the overall system architecture. Chapter 5 specifies the automata model and the YAML DSL. Chapter 6 covers the C++ runtime engine internals. Chapter 7 describes the gateway and communication protocol. Chapter 8 covers the IDE. Chapter 9 describes the Petri net analysis layer. Chapter 10 presents the showcase catalog. Chapter 11 concludes with results and future work.

Chapter 2

Background

2.1 Finite State Machines

A *Finite State Machine* (FSM) is a well-established mathematical formalism for modeling systems that can be in exactly one of a finite number of states at any given time and that change state in response to inputs [6]. Formally, a deterministic FSM is defined as a 5-tuple:

$$M = (Q, \Sigma, \delta, q_0, F) \tag{2.1}$$

where:

- Q is a finite, non-empty set of states,
- Σ is a finite input alphabet (set of events or signals),
- $\delta : Q \times \Sigma \to Q$ is the transition function,
- $q_0 \in Q$ is the initial state,
- $F \subseteq Q$ is the set of accepting (final) states.

In control applications the acceptance criterion is typically not used; the focus lies instead on the transition function and any actions associated with states or transitions. Two classical variants are distinguished:

Moore machine Output depends only on the current state. Output is produced upon entering a state.

Mealy machine Output depends on both the current state and the current input. Output is produced when a transition fires.

Aetherium Automata supports both semantics simultaneously through per-state code hooks executed on entry and per-tick, and per-transition code executed when a transition fires.

2.2 Extended Finite State Machines

A plain FSM is insufficient for most practical control problems because all variation must be encoded into distinct states. An *Extended Finite State Machine* (EFSM) augments

the FSM with a variable store, transforming guards and actions into functions over that store [5]:

$$M_E = (Q, \Sigma, V, \delta_E, \lambda, q_0) \tag{2.2}$$

where:

- Q, Σ, q_0 are as in the standard FSM,
- V is a finite set of typed variables (the *data context*),
- $\delta_E : Q \times \Sigma \times Bool(V) \to Q$ is the extended transition function; each transition edge carries a *guard* $g : V \to Bool$ that must be satisfied for the transition to fire,
- $\lambda : Q \times \Sigma \times V \to V$ is the output/action function that updates the variable store when a transition fires.

In Aetherium, the variable store V contains three classes of variables: *input* variables (written by the external environment, read-only to the automaton), *output* variables (written by the automaton, consumed externally), and *internal* variables (read-write by the automaton alone). Guards and actions are expressed as Lua [8, 7] scripts evaluated against V at runtime. Lua was specifically designed as a lightweight, embeddable extension language [8], making it well suited for use inside a portable C++ runtime with constrained memory budgets.

Each state $q \in Q$ additionally carries three optional code hooks evaluated in the following order:

1. `on_enter` — executed once when the state is entered,
2. `body` — executed on every execution tick while the machine remains in the state,
3. `on_exit` — executed once immediately before leaving the state.

This structure corresponds to a Moore/Mealy hybrid: `on_enter` provides state-dependent output (Moore), while the transition `body` hook provides input-dependent output (Mealy).

2.3 Petri Nets

A *Petri net* is a mathematical formalism designed for modeling concurrent, distributed, and asynchronous systems [15]. It generalizes finite automata by allowing multiple simultaneous active conditions (tokens in multiple places). A formal survey with analysis properties is given by Murata [11]. A Petri net is defined as a 4-tuple:

$$N = (P, T, F, M_0) \tag{2.3}$$

where:

- P is a finite set of *places* (representing conditions or states),
- T is a finite set of *transitions*, with $P \cap T = \emptyset$,
- $F \subseteq (P \times T) \cup (T \times P)$ is the *flow relation* (directed arcs),
- $M_0 : P \to \mathbb{N}$ is the *initial marking* (token distribution).

A transition $t \in T$ is *enabled* under marking M if every input place p of t holds at least one token: $\forall p \in \bullet t : M(p) \geq 1$. Firing t removes one token from each input place and adds one token to each output place:

$$M'(p) = M(p) - F(p,t) + F(t,p) \tag{2.4}$$

Petri nets provide the foundation for several analysis problems relevant to IoT networks:

Reachability Determine whether a given marking M' is reachable from M_0.

Boundedness Determine whether the number of tokens in any place is bounded.

Liveness (deadlock-freedom) Determine whether every transition can eventually fire from every reachable marking.

In Aetherium, the Petri net formalism is applied at the *network level*: each automaton's current state contributes a conceptual token, and the combined network of automata is analyzed for deadlocks, bottlenecks, and resource contention. This is described in detail in Chapter 9.

Chapter 3

Analysis of Existing Solutions

Before designing Aetherium Automata, two representative existing platforms were analysed: Node-RED, which represents the flow-based IoT automation category, and Eclipse 4diac, which represents the IEC 61499 function-block category. Together they cover the main points of comparison relevant to the thesis goals: explicit state modelling, embedded portability, testability, and observability.

3.1 Node-RED

3.1.1 Overview

Node-RED [13] is a low-code, flow-based programming tool originally developed in early 2013 by Nick O'Leary and Dave Conway-Jones at IBM's Emerging Technology Services group. It was open-sourced in September 2013 and later donated to the OpenJS Foundation (through the JS Foundation in 2016). Today it is one of the most widely adopted visual programming environments for IoT prototyping.

3.1.2 Programming Model

Node-RED implements the *flow-based programming* paradigm [12], in which an application is expressed as a directed graph of *nodes* connected by *wires*. Each node encapsulates a single unit of processing and communicates with adjacent nodes by passing JavaScript message objects (`msg`). Flows are the top-level grouping unit and are serialised as JSON. The browser-based editor communicates with a Node.js runtime that executes the flows.

Three kinds of built-in nodes are provided:

Input nodes Inject messages into the flow from external sources — timers, MQTT brokers, HTTP endpoints, or hardware GPIO.

Processing nodes Transform, filter, or route messages — function nodes accept arbitrary JavaScript code, switch nodes branch on message properties.

Output nodes Send messages to external sinks — MQTT publish, HTTP response, database write.

A community ecosystem of over 5 000 third-party nodes extends the platform with protocol adapters, UI widgets, and cloud service integrations.

3.1.3 State Management

Node-RED has no built-in concept of a state machine. Application-level state must be managed explicitly using *context storage*: node context (local to one node), flow context (shared within a flow tab), or global context (shared across all flows). Accessing and mutating context from within function nodes requires manual bookkeeping and is untyped.

Several community packages attempt to address this gap by adding FSM nodes (e.g. `node-red-contrib-finite-statemachine`, `node-red-contrib-xstate-machine`), but these are third-party add-ons with no native IDE integration, no visual state graph, and no support for timed or probabilistic transitions. Representing complex stateful logic natively in Node-RED leads to large, hard-to-navigate flow graphs in which control flow and data flow are interleaved.

3.1.4 Embedded Targets and Observability

Node-RED runs on the Node.js runtime, which requires a relatively capable host: minimum a Raspberry Pi class device with a full operating system. Bare-metal microcontrollers such as ESP32 (without an OS) or ARM Cortex-M targets are not directly supported.

Runtime observability is provided through a Debug side-panel that displays message values at selected wire taps. There is no structured execution trace, no replay capability, and no fault injection mechanism. Reproducing a timing-sensitive failure requires either manual test setup or third-party logging infrastructure.

3.1.5 Summary

Node-RED excels at rapid prototyping and data routing in environments where state is simple or absent. It is poorly suited to applications requiring rich stateful behaviour, reproducible testing under adverse conditions, or deployment on constrained embedded hardware.

3.2 Eclipse 4diac

3.2.1 Overview

Eclipse 4diac [3] (originally named 4DIAC — Framework for Distributed Industrial Automation and Control) is an open-source development environment and runtime for distributed control systems based on the IEC 61499 standard. It was first presented by Strasser, Rooker, Zoitl et al. in 2008 [18] and later became an Eclipse Foundation project. Its primary reference text is the book by Zoitl and Strasser [22].

3.2.2 IEC 61499 and Function Blocks

IEC 61499 is an international standard for distributed industrial process measurement and control systems. It defines a *Function Block* (FB) as the primary computational unit and extends the earlier IEC 61131-3 PLC programming standard to multi-device, distributed architectures.

Three FB types are defined:

Basic Function Block (BFB) The fundamental executable unit. Its behaviour is defined by an *Execution Control Chart* (ECC) — an event-driven state machine —

combined with one or more *algorithms* expressed in Structured Text, Ladder Diagram, or Function Block Diagram (languages drawn from IEC 61131-3). A BFB transitions between ECC states when input *events* arrive and associated Boolean guard conditions are satisfied. Executing an ECC state fires the associated algorithm, which may produce output events and set output data variables.

Composite Function Block (CFB) A network of interconnected FBs, providing hierarchical composition without a separate ECC. The internal topology is fixed at design time.

Service Interface Function Block (SIFB) Provides a typed interface to external services such as hardware peripherals, network protocols, or operating system calls.

The ECC of a BFB is structurally similar to an EFSM (Section 2.2): states, guarded transitions, and per-state algorithms. However, the trigger mechanism differs fundamentally — transitions fire only in response to named input *events* (discrete signals), not as a result of continuous condition polling or timer expiry. Timed behaviour must be implemented using dedicated timer SIFBs wired into the network, and probabilistic transitions are not part of the standard.

3.2.3 4diac IDE and FORTE Runtime

The **4diac IDE** is based on the Eclipse framework. It provides a graphical editor for composing FB networks, an ECC editor for designing BFB state machines, and deployment tooling for distributing application partitions across multiple devices.

The **4diac FORTE** runtime is a portable C++ implementation of the IEC 61499 execution model targeting 16-bit and 32-bit embedded devices. It supports real-time execution and online reconfiguration of running applications. Supported platforms include Linux, Windows, VxWorks, and several bare-metal embedded targets.

3.2.4 Limitations Relevant to this Thesis

Despite its rigorous foundations, 4diac exhibits several limitations from the perspective of the Aetherium design goals:

- **Event-only trigger model.** The ECC transitions fire exclusively on named input events. There is no native support for time-based transitions (without auxiliary timer FBs), no probabilistic selection, and no direct reactivity to data threshold crossings without additional logic.
- **Scripting.** Algorithms are written in IEC 61131-3 languages (primarily Structured Text). There is no embedded general-purpose scripting language comparable to Lua. Dynamic behaviour that depends on runtime arithmetic requires either pre-compiled C++ SIFBs or complex FB networks.
- **IDE weight.** The Eclipse-based IDE carries a significant installation footprint and a steep learning curve, in contrast to a purpose-built Electron application.
- **No fault injection or replay.** 4diac provides no mechanism to inject network delays, packet loss, or disconnection events into a running deployment, and no execution trace suitable for step-by-step replay.

- **Informal execution semantics.** The original IEC 61499 standard left execution semantics underspecified, leading to interpreter divergence across implementations. While later editions improved this, the ECC execution model remains more complex to reason about than the deterministic tick-based resolution used in Aetherium.

3.3 Comparison

Table 3.1 summarises the key characteristics of both analysed platforms against Aetherium Automata.

Table 3.1: Feature comparison of Node-RED, Eclipse 4diac, and Aetherium Automata.

Feature	Node-RED	4diac	Aetherium
Explicit state model	No	Yes (ECC)	Yes (EFSM)
Transition types	N/A	Event + guard	Classic, Timed, Event, Probabilistic, Immediate
Scripting language	JavaScript	Structured Text	Lua 5.4
Visual editor	Browser (Node.js)	Eclipse IDE	Electron desktop
Embedded runtime	No	Yes (FORTE, C++)	Yes (C++17)
Embedded targets	Raspberry Pi+	16/32-bit MCUs	ESP32, Pico, MCXN947
Flow/program format	JSON	XML (FBT files)	YAML
Typed variables	No (JavaScript)	Yes (IEC 61131-3)	Yes (7 types)
Fault injection	No	No	Yes
Time-travel debugging	No	No	Yes
Structural analysis	No	No	Petri net analysis
Target domain	General IoT	Industrial automation	General IoT + embedded

The analysis reveals that neither platform simultaneously satisfies all of the requirements identified in Section 3.4. Node-RED lacks explicit state modelling entirely and cannot run on bare-metal embedded targets. 4diac provides formal state machines through the ECC but restricts transitions to an event-driven model and provides no fault injection or replay capability. Aetherium Automata addresses these gaps by combining a five-type EFSM model, a portable C++ runtime, deterministic fault injection, and execution trace replay within a single unified toolchain.

3.4 Requirements

Based on the analysis above and the thesis goals, the following requirements were identified:

R1 – EFSM expressiveness The model must support guarded transitions, time-driven transitions, event/signal-driven transitions, probabilistic transitions, and unconditional (epsilon) transitions.

R2 – Scripting Guards and actions must be expressible as scripts evaluated against the current variable store. Lua is chosen as the scripting language.

R3 – Portability The runtime engine must compile and execute on desktop Linux/macOS, ESP32 (Arduino/FreeRTOS), Raspberry Pi Pico, and ARM Cortex-M (MCXN947).

R4 – DSL Automata definitions must be authored in a human-readable format. YAML is chosen.

R5 – Real-time monitoring The IDE must display the current state and variable values of running automata with low latency.

R6 – Fault injection The system must support deterministic fault injection (delay, jitter, drop, disconnect) for testing.

R7 – Time-travel debugging Execution traces must be recorded and replayable step-by-step in the IDE.

R8 – Structural analysis The system must detect deadlocks and bottlenecks in multi-automata networks using Petri net analysis.

Chapter 4

System Architecture

The Aetherium Automata platform is composed of four major components that form a layered architecture:

1. **Engine** — a portable C++17 library that parses, loads, and executes EFSM definitions.
2. **Server** — an Elixir/OTP application that manages connected devices, routes messages, and aggregates state.
3. **Gateway** — an Elixir/Phoenix application that exposes a WebSocket channel interface to the IDE.
4. **IDE** — an Electron desktop application (TypeScript/React) for visual design, deployment, and monitoring.

Figure 4.1 shows the high-level component topology.

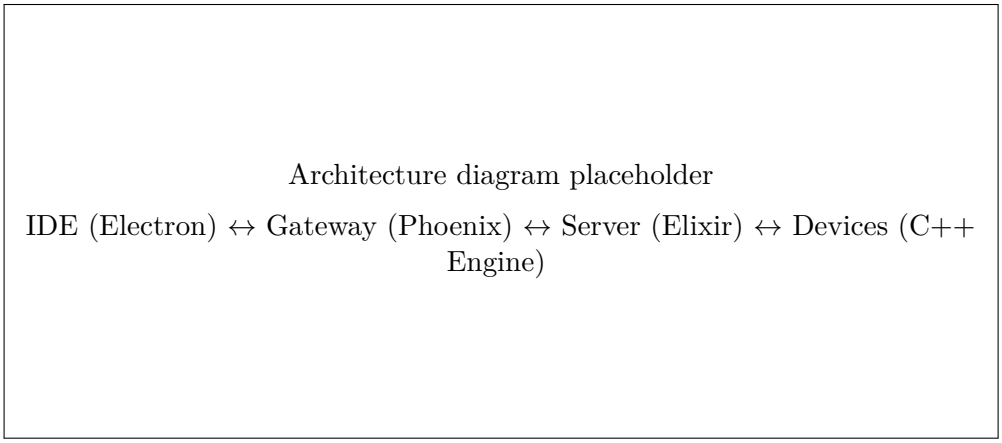

Figure 4.1: High-level component architecture of Aetherium Automata.

4.1 Component Responsibilities

4.1.1 Engine

The engine is the core library. It is written in C++17 with no dynamic dependencies beyond the platform-specific backends (Lua 5.4 [7] on desktop, IXWebSocket [9] for transport). It is responsible for:

- parsing automata definitions from YAML [21] (via RapidYAML [2]),
- maintaining the variable store and evaluating Lua guards and actions (via Sol2 [19]),
- resolving and firing transitions on each execution tick,
- transmitting telemetry and receiving commands over a pluggable transport interface.

Three abstract interfaces isolate all platform-specific behavior:

`IClock` Monotonic time source. Desktop: `std::chrono`. Embedded: hardware timer peripheral.

`IRandomSource` Pseudorandom number generator. Desktop: `std::mt19937_64`. Embedded: hardware RNG.

`IScriptEngine` Script evaluation. Desktop: Lua 5.4 via Sol2. Embedded: lightweight evaluator or no-op stub.

CMake build targets select the appropriate implementations at compile time.

4.1.2 Server

The server is an Elixir/OTP application. It manages the registry of connected devices, buffers messages, routes inputs to devices and outputs to subscribers, and handles deployment commands. It communicates with devices over WebSocket, MQTT, or Serial/USB depending on the device type.

4.1.3 Gateway

The gateway is an Elixir/Phoenix application that exposes a *Phoenix Channel* endpoint. The IDE connects to this endpoint over a persistent WebSocket. The gateway translates IDE commands (deploy, start, stop, set input) into server calls, and streams device events (state changes, variable updates, telemetry) back to the IDE in real time.

4.1.4 IDE

The IDE is an Electron desktop application. The main process handles native file system access. The renderer process is a React single-page application with Zustand-based state management. The IDE communicates with the gateway exclusively via Phoenix Channels.

4.2 Deployment Model

Devices are deployed using Docker Compose for local multi-service orchestration. A typical local stack consists of the gateway service, the server service, one or more device emulator services (running the engine on desktop), and optionally a black-box probe container for sandboxed external systems.

Chapter 5

Automata Model and DSL

5.1 Model Overview

An *automaton* in Aetherium is a named, versioned EFSM definition. At the top level it contains:

- **config** — name, version, layout type (`inline` or `folder`), description, author, tags,
- **variables** — typed I/O and internal variables,
- **automata** — the EFSM body: initial state, states, and transitions.

5.2 States

Each state $q \in Q$ is identified by a string `StateId` matching the pattern `^[A-Za-z_][A-Za-z0-9_]*$`. States carry three optional Lua code hooks:

`on_enter` Executed once when the machine transitions into this state.

`body` Executed on every execution tick while the machine remains in this state.

`on_exit` Executed once immediately before transitioning away.

One state per automaton is designated the `initial_state`; the engine enters it when execution begins.

5.3 Variables

Variables form the data context V of the EFSM. Each variable is defined by a *VariableSpec* that carries:

- **name** — unique string identifier,
- **type** — one of `bool`, `int32`, `int64`, `float32`, `float64`, `string`, `binary`, `table`,
- **direction** — `input` (read-only, written by the external environment), `output` (write-only for the environment, written by the automaton), or `internal` (read-write by the automaton),

- **default** — optional initial value.

At runtime each variable is a *Variable* instance wrapping a type-safe `Value` variant and a boolean `changed()` flag. The flag is set on every write and cleared explicitly or on state transition. It is used by event-type transitions for O(1) reactive detection without polling.

5.4 Transition Types

Aetherium defines five transition kinds, each with distinct enabling semantics:

5.4.1 Classic

A classic transition is enabled when a Lua guard expression evaluates to `true`:

$$\text{enabled}(t) \iff g_t(V) = \mathit{true} \tag{5.1}$$

Guards are arbitrary Lua expressions operating on the variable store through the engine API.

5.4.2 Timed

A timed transition fires based on time domain conditions. Five modes are supported:

`after` Fire after a fixed delay Δt (in milliseconds) from the last state entry.

`every` Fire repeatedly with period τ.

`timeout` Fire if no other transition from the same state has fired within Δt.

`at` Fire at an absolute timestamp.

`window` Fire only if the current time lies within $[t_{start}, t_{end}]$.

An optional jitter parameter $\varepsilon \geq 0$ adds a uniform random offset to the scheduled fire time:

$$t_{fire} = t_{target} + \mathcal{U}(0, \varepsilon) \tag{5.2}$$

This prevents synchronized firing across multiple automata instances and models real-world timer imprecision.

5.4.3 Event

An event transition is enabled when a named variable satisfies a specified signal condition. Supported trigger types are:

`on_change` Any write to the variable since the last evaluation.

`on_rise` The variable transitions from `false` to `true`.

`on_fall` The variable transitions from `true` to `false`.

`on_threshold` The variable crosses a comparator threshold: $v \bowtie k$ where $\bowtie \in \{>, <, \geq, \leq, =, \neq\}$.

An optional debounce delay δ_d filters spurious signals shorter than the threshold. The `one_shot` flag causes a threshold trigger to fire at most once per crossing. A transition may specify multiple triggers, combined with either AND or OR logic (`require_all`).

5.4.4 Probabilistic

A probabilistic transition is always structurally enabled. Its *weight* $w_i \in [0, 10000]$ (representing 0.00%–100.00% in fixed-point) participates in a weighted roulette selection among all enabled transitions in the same priority group. The selection probability is:

$$P(T_i) = \frac{w_i}{\sum_{j \in G} w_j} \tag{5.3}$$

where G is the set of enabled transitions in the highest-priority group. Weights may be Lua expressions re-evaluated on each tick, enabling dynamic probability distributions.

5.4.5 Immediate (Epsilon)

An immediate transition is always enabled and is resolved before transitions of any other type. It models the ε-transitions of non-deterministic finite automata: unconditional, instantaneous state changes used to chain states without observable intermediate steps.

5.5 Transition Resolution Algorithm

On each execution tick the engine resolves which transition, if any, fires from the current state. The algorithm is:

1. Collect all outgoing transitions T_{out} from the current state q.
2. Evaluate each transition for enablement according to its type (Section 5.4).
3. Let $E \subseteq T_{out}$ be the set of enabled transitions.
4. If $E = \emptyset$: remain in q, execute `body`, terminate tick.
5. Sort E by priority value (ascending integer; lower value = higher priority).
6. Let $G \subseteq E$ be the subset with the minimum (highest) priority value.
7. Select a transition $t^* \in G$:
   (a) If $|G| = 1$: $t^* \leftarrow$ the single element.
   (b) If all members of G have weight > 0: apply Equation (5.3).
   (c) Otherwise: $t^* \leftarrow$ the first element in declaration order (deterministic fallback).
8. Fire t^*:
   (a) Execute `on_exit` of q.
   (b) Execute the transition `body` hook of t^*.
   (c) Move to target state $q' = \delta_E(q, t^*)$.
   (d) Execute `on_enter` of q'.

Priority establishes a total deterministic order; probabilistic selection applies only within a single priority group. This cleanly separates intentional non-determinism from deterministic control flow.

5.6 Black-Box Contract

An automaton may optionally declare a *black-box contract* — a formal interface specification for external systems. The contract defines:

- **ports** — named inputs/outputs with direction, type, observability flag (visible to monitors), and fault-injectability flag,
- **emitted events** — names of events the automaton produces,
- **observable states** — the subset of states visible to external monitors,
- **resources** — shared or exclusive resources with capacity and latency sensitivity annotations.

Black-box contracts are used by the structural analysis layer (Chapter 9) and by the fault injection subsystem.

5.7 YAML Domain-Specific Language

Automata are authored in YAML [21]. The top-level schema is:

```yaml
version: 0.0.1
config:
  name: <string>
  type: inline | folder
  location: <path> # required if type: folder
  language: lua # optional, default lua
  description: <string> # optional
  author: <string> # optional
  tags: [<string>, ...] # optional

variables:
  - name: <VarName>
    type: bool | int32 | int64 | float32 | float64 | string | binary
    direction: input | output | internal
    default: <value> # optional

automata:
  initial_state: <StateId>
  states:
    <StateId>:
      on_enter: |
        <lua code>
      body: |
        <lua code>
      on_exit: |
        <lua code>
  transitions:
    <TransitionId>:
      from: <StateId>
      to: <StateId>
      type: classic | timed | event | probabilistic | immediate
      priority: <int> # lower value = higher priority
```

```
      weight: <float> # for probabilistic selection
      condition: | # for type: classic
        <lua expression>
      timed: # for type: timed
        mode: after | at | every | timeout | window
        after: <ms>
        jitter: <ms> # optional
      event: # for type: event
        triggers:
          - signal: <VarName>
            trigger: on_change | on_rise | on_fall | on_threshold
            threshold:
              operator: ">" | "<" | ">=" | "<=" | "==" | "!="
              value: <number>
              one_shot: <bool>
        require_all: <bool>
```

Listing 5.1: YAML DSL top-level structure.

5.7.1 Layout Types

The `inline` layout places all state and transition Lua code directly inside the YAML file. The `folder` layout externalizes code into separate files referenced by path, enabling better IDE tooling, version-control diffs, and code reuse.

5.7.2 Validation Rules

The parser enforces the following constraints:

- All state and transition identifiers must match `^[A-Za-z_][A-Za-z0-9_]*$`.
- Every transition endpoint (`from`, `to`) must reference a state defined in `automata.states`.
- Variable names must be unique within an automaton.
- Exactly one state must be designated `initial_state`.
- The `timed.mode` field determines which time parameters are required.

Chapter 6

Runtime Engine

6.1 Engine Architecture

The engine is structured as a set of cooperating modules within a single C++17 static library (`aetherium_runtime_core`). The principal modules are:

`model.hpp` Data types: `State`, `Transition`, `Automata`, `CodeBlock`, and per-type transition configuration structs (`ClassicConfig`, `TimedConfig`, `EventConfig`, `ProbabilisticConfig`).

`variable.hpp` `Value` (a type-safe variant over `bool`, `int32_t`, `int64_t`, `float`, `double`, `string`, `vector<uint8_t>`), `VariableSpec`, `Variable` (runtime instance with `changed()` flag and timestamp), `VariableStore`.

`types.hpp` Enumerations (`ValueType`, `TransitionType`, `TimedMode`, `EventTrigger`, `CompareOp`, `ExecutionState`), ID types (`StateId`, `TransitionId`, `VariableId`), and a `Result<T>` monad for error propagation.

`parser.hpp/cpp` YAML [21] deserialization via RapidYAML [2] (`ryml`). Produces a validated `Automata` model.

`runtime.hpp/cpp` The execution loop. Drives the tick cycle, manages per-transition timer entries, and dispatches the transition resolution algorithm.

`engine.cpp/hpp` The top-level orchestrator. Receives commands from the transport layer, dispatches to the runtime, and produces telemetry events.

`lua_engine.cpp/hpp` Lua 5.4 [7] scripting host via Sol2 [19]. Exposes the engine API to scripts (Section 6.2).

`transport.hpp` Abstract `ITransport` interface. `WebSocketTransport` implements it via IXWebSocket [9].

`protocol.cpp/hpp` Binary wire format serialization and deserialization.

`execution_trace.hpp` `TraceRecord` (per-event structured record), `FaultProfile` (injection parameters), `DeploymentDescriptor` (device metadata).

`telemetry_log_hub.cpp/hpp` Structured log event capture and streaming callbacks.

`artifact.hpp/cpp` Binary container format for deployable automata.

6.2 Lua Scripting

Guards and state/transition code are authored in Lua 5.4 [8, 7]. The engine exposes the following global API functions to scripts:

`value(name)` Read the current value of a variable by name.

`setVal(name, v)` Write a value to a variable by name.

`check(name)` Return `true` if the variable's `changed()` flag is set.

`emit(name)` Signal a named event.

`log(msg)` Write a message to the telemetry log.

`rand()` Return a uniform random float in $[0, 1)$ using the platform `IRandomSource`.

`now()` Return the current monotonic timestamp in milliseconds from `IClock`.

`clamp(v, lo, hi)` Clamp v to $[lo, hi]$.

On platforms where full Lua is not available (highly constrained microcontrollers), a `SimpleScriptEngine` or no-op stub can be substituted via the `IScriptEngine` interface without modifying any other engine module.

6.3 Timer Management

The runtime maintains a table of timer entries, one per timed transition. Each entry stores:

- `startTime` — the monotonic time at which the timer was armed,
- `targetTime` — the computed fire time (including jitter),
- `repeatCount` — remaining repetitions for `every` mode (0 = infinite).

On each tick the runtime queries `IClock::now()` and compares it against `targetTime` for each armed timer. Expired timers with `mode = every` are immediately re-armed with a new `targetTime = now + period + jitter`.

6.4 Fault Injection

Fault injection is a proven technique for testing the resilience of distributed systems under realistic adverse conditions [1]. A `FaultProfile` can be attached to a deployment descriptor per device. The profile specifies:

Ingress faults Applied to messages arriving at the device:

- fixed delay and jitter (ms),
- drop probability $p_d \in [0, 1]$,
- duplicate probability $p_{dup} \in [0, 1]$.

Egress faults The same parameters applied to outgoing messages.

Disconnect simulation Periodic forced disconnection windows of configurable duration.

Battery drain Per-tick and per-message energy consumption, combined with a `BatteryProfile` that models charge state and cutoff voltage.

All random decisions within the fault injector use the platform `IRandomSource` seeded from a field in the `FaultProfile`, ensuring deterministic and reproducible fault scenarios across runs.

6.5 Execution Trace and Time-Travel Debugging

Record-and-replay is a well-established strategy for debugging distributed systems: events are recorded during execution and can be replayed deterministically to reproduce failures [4]. Aetherium applies this principle at the automata level. Every engine event is recorded in an `ExecutionTrace` as a sequence of `TraceRecord` entries. Each record carries:

- sequence number,
- wall-clock and monotonic timestamps,
- event type (state transition, variable change, message send/receive),
- old and new values,
- latency measurements,
- the fault actions applied to the associated message (if any).

The trace is exported to newline-delimited JSON (JSON Lines) for post-mortem tooling. The IDE's runtime view panel can load a trace and step forward and backward through it, reconstructing the exact variable and state values at each recorded instant. This enables deterministic replay of timing-sensitive bugs in distributed systems without requiring hardware reproduction of the fault.

Chapter 7

Gateway and Communication Protocol

7.1 Binary Wire Protocol

All messages between the engine and the server use a compact binary envelope:

$$\underbrace{\texttt{0xAE01}}_{\text{magic}}\ \underbrace{\texttt{0x01}}_{\text{version}}\ \underbrace{\texttt{type}}_{\text{1 byte}}\ \underbrace{\texttt{length}}_{\text{4 bytes}}\ \underbrace{\texttt{payload}}_{\text{N bytes}} \tag{7.1}$$

Messages are organized into three categories:

Control plane `Hello`, `Ping`, `Pong`, `Status` — connection management.

Automata plane `LoadAutomata`, `Start`, `Stop`, `Restart` — lifecycle commands.

Data plane `Input`, `Output`, `StateChange`, `Telemetry` — runtime data exchange.

Large automata definitions that exceed the message MTU are fragmented using a chunked-transfer sub-protocol before being reassembled on the receiver side.

7.2 Phoenix Gateway

The gateway is implemented as an Elixir [20]/Phoenix [16] application using Phoenix Channels. Phoenix Channels provide persistent, topic-routed bidirectional WebSocket connections with presence tracking. Each IDE client session maps to one channel subscription.

The gateway exposes the following operations to the IDE:

- `ping()` — test channel connectivity,
- `listDevices()` — retrieve the current device registry,
- `deployAutomata(yaml, deviceId)` — transmit an automaton definition to a device,
- `startAutomata(deviceId, runId)` — command a device to begin execution,
- `stopAutomata(deviceId, runId)` — command a device to halt,
- `setInput(deviceId, varName, value)` — write an input variable,

- `getSnapshot()` — retrieve the current full device/variable state.

Snapshot caching reduces reconnection latency: when the IDE reconnects, it receives the last cached snapshot rather than waiting for a full device re-advertisement cycle.

7.3 End-to-End Message Flow

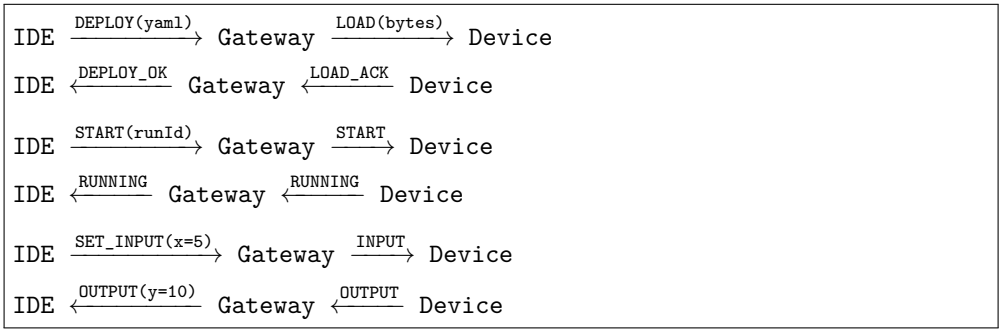

Figure 7.1: Simplified end-to-end message flow for deploy, start, and I/O exchange.

Chapter 8

Integrated Development Environment

8.1 Architecture

The IDE is built on Electron [14], which embeds a Node.js main process and a Chromium-based renderer process. The main process exposes native file system access and IPC bridges. The renderer process is a React [10] single-page application.

State management uses Zustand [17], a lightweight library that exposes individual atomic stores with direct mutation. Each store covers a narrow concern:

Table 8.1: Zustand stores in the IDE renderer.

Store	Responsibility
`automataStore`	Current automaton definition being edited
`projectStore`	Multi-automaton project and file management
`runtimeViewStore`	Live visualization state (current state, variable values, zoom)
`gatewayStore`	Phoenix channel connection, device list, pending commands
`analyzerStore`	Petri net analysis results (deadlocks, bottlenecks)

8.2 Phoenix Gateway Service

`PhoenixGatewayService` is the IDE-side client for the Phoenix Channel. It manages the WebSocket lifecycle (connect, reconnect, presence), serializes commands into channel push events, and dispatches received events to the appropriate Zustand store actions. Command outcome tracking allows the UI to display pending/success/error states per operation.

8.3 Panels

The IDE UI is organized into panels:

Canvas / State Machine Editor Visual graph editor. States are rendered as nodes; transitions as directed edges annotated with type, priority, and weight. Users can drag to create states, draw transitions, and edit properties inline.

State Inspector Edits `on_enter`, `body`, and `on_exit` Lua code for the selected state with syntax highlighting.

Transition Inspector Edits condition, type, priority, weight, and type-specific parameters (timed mode, event triggers) for the selected transition.

Variables Panel Unified view of all input, output, and internal variables with current values and types.

Gateway Panel Connection settings, device list, and manual command testing.

Runtime Monitor Panel Live display of the current state (highlighted in the canvas) and variable values during execution.

Log Panel Streaming telemetry event log with filtering by level and source.

Network Panel Topology view of the multi-device network and inter-automaton wiring.

Analyzer Panel Petri net analysis output: bottleneck warnings, deadlock candidates, contention reports.

Petri Net Panel Visual representation of the Petri net derived from the automata network.

Explorer Panel File-based project navigator for folder-layout automata.

Chapter 9

Petri Net Analysis

9.1 Lifting Automata to Petri Nets

A network of communicating automata can be analyzed structurally by lifting it to a Petri net representation. Each automaton's current state contributes a token to a corresponding place. The communication channels between automata are modeled as arcs connecting output transitions of one automaton's places to input places of another.

This lifting is performed statically from the YAML definitions, without requiring execution: the analyzer reads all automata in the project, maps each state to a place, maps each transition to a Petri net transition, and wires inter-automaton signal connections as shared places.

9.2 Analysis Operations

The analyzer performs the following checks:

Deadlock detection A marking M is a deadlock if no transition in the Petri net is enabled under M. The analyzer reports states of the automata network from which no further progress is possible.

Unbounded place detection A place is unbounded if tokens can accumulate without limit. In an automata network this corresponds to an output channel that is written faster than it is consumed—a queue overflow risk.

Bottleneck identification Places that are persistently marked across the reachability graph indicate points of sustained contention or backpressure.

Contention analysis Resources declared in black-box contracts (Section 5.6) are modeled as bounded-capacity places. The analyzer identifies pairs of automata that may simultaneously request the same exclusive resource and reports potential mutual exclusion violations.

9.3 Showcase Scenarios

Two showcase categories directly exercise the Petri net analysis:

Showcase 13 – Petri Signal Chain A four-automaton pipeline: command router → safety gate → drive unit → telemetry observer. The analyzer visualizes the token flow and identifies whether the safety gate can become a bottleneck under high command rates.

Showcase 14 – Petri Contention Three automata (charger node, motion axis, power allocator) competing for a shared power budget. The analyzer detects the potential contention and reports which automata pairs require arbitration.

Chapter 10

Showcase Catalog

The system is validated by a catalog of 18 showcase automata organized into 14 categories. All definitions reside in `example/automata/showcase/` and are verified by `example/automata/showcase/`

Table 10.1: Showcase automata catalog.

#	Category	Key concepts demonstrated
01	Basics	Timed and classic transitions, manual override input
02	Control	Event threshold triggers, pump state control
03	Probabilistic	Weighted 3-lane quality routing
04	Resilience	Watchdog timer, fault detection, retry with backoff
05	Energy	Load prediction, peak shaving scheduler
06	Pipeline	Multi-stage line buffer dispatcher
07	Folderized	Folder-based code layout, door safety controller
08	ESP32	PWM, LED, OLED, thermostat, production line (8 automata)
09	MCXN947	ARM Cortex-M7: GPIO, buttons, touch, temperature (4 automata)
10	Guarded Cell	6-automata safety supervision network
11	Bidirectional Loop	4 automata with bidirectional signal wiring
12	Black Box	Docker-sandboxed probe, external system wrapping
13	Petri Signal Chain	Command router → safety gate → drive unit → telemetry
14	Petri Contention	Charger, motion axis, power allocator competing for resources

10.1 Selected Showcase: Quality Router (03)

The quality router demonstrates probabilistic transition selection. Three output lanes (A, B, C) are represented as transitions from a single routing state. Each transition carries a weight:

- Lane A: weight 5000 (50%)
- Lane B: weight 3000 (30%)
- Lane C: weight 2000 (20%)

On each tick the transition resolver applies Equation (5.3) to select the lane. The weights are static in this example but could be Lua expressions that respond to sensor readings (e.g., routing more batches to Lane A when Lane B is congested).

10.2 Selected Showcase: Sensor Watchdog Recovery (04)

The watchdog demonstrates timed transitions combined with classic guarded recovery. The automaton cycles through:

1. `monitoring` — waiting for sensor data with a timeout transition.
2. `alarming` — entered when the timeout fires (sensor silent), emitting an alert output.
3. `recovering` — attempting to restart the sensor, with a timed retry and a maximum retry count tracked in an internal variable.

This pattern is a common real-world requirement: detect device failure, raise an alarm, attempt structured recovery, and escalate if recovery fails.

Chapter 11

Conclusion

11.1 Summary

This thesis presented Aetherium Automata, a platform for the visual design, deployment, and monitoring of distributed IoT control systems modeled as Extended Finite State Machines. The system was designed around the principle that stateful control logic is better expressed explicitly as a state machine than embedded in ad-hoc scripts or workflow engines.

The key contributions are:

1. A formal EFSM model supporting five transition types (classic, timed, event, probabilistic, immediate), typed variables with reactive change detection, and Lua-scripted guards and actions.
2. A YAML domain-specific language for authoring automata in both inline and folder layouts, with a parser that validates structure and identifier correctness.
3. A portable C++17 runtime engine that executes the model on desktop Linux and embedded platforms (ESP32, Raspberry Pi Pico, MCXN947) via three abstract platform interfaces (`IClock`, `IRandomSource`, `IScriptEngine`).
4. A binary wire protocol and Elixir/Phoenix gateway for real-time, bidirectional IDE-to-device communication.
5. An Electron IDE with live state monitoring, variable inspection, fault injection, and time-travel debugging via execution trace replay.
6. A Petri net analysis layer that detects deadlocks, bottlenecks, and resource contention in multi-automata networks.
7. A catalog of 18 showcase automata covering real-world IoT scenarios including resilience, energy management, probabilistic routing, and multi-device coordination.

11.2 Future Work

The following directions are identified for future development:

- Full Petri net reachability analysis (currently the analyzer performs structural checks; complete reachability graph construction is pending).

- Bytecode compilation of Lua scripts into a compact `EngineBytecodeProgram` format for deployment to the most constrained targets.
- Hierarchical (nested) automata to support modular composition of complex behaviors.
- Extended IDE canvas: undo/redo history, multi-select, copy-paste of states and transitions.
- Integration with InfluxDB and Grafana for long-running time-series metrics dashboards.

Bibliography

[1] Beilharz, J.; Wiesner, P.; Boockmeyer, A.; Friedenberger, D.; Brokhausen, F. et al. Continuously Testing Distributed IoT Systems: An Overview of the State of the Art. In: *Service-Oriented Computing – ICSOC 2021 Workshops*. Cham: Springer, 2022, p. 375–387.

[2] Biga, J. P. M. *RapidYAML: Rapid YAML — a library to parse and emit YAML, and do it fast* online. 2024. Available at: https://github.com/biojppm/rapidyaml. [cit. 2025-05-11]. MIT License.

[3] Eclipse Foundation. *Eclipse 4diac: Open Source Infrastructure for Distributed Industrial Control Based on IEC 61499* online. 2024. Available at: https://eclipse.dev/4diac/. [cit. 2025-05-11].

[4] Geels, D.; Altekar, G.; Shenker, S. and Stoica, I. Replay Debugging for Distributed Applications. In: *2006 USENIX Annual Technical Conference (USENIX ATC 06)*. Boston, MA: USENIX Association, 2006, p. 289–300. Available at: https://www.usenix.org/conference/2006-usenix-annual-technical-conference/replay-debugging-distributed-applications. Best Paper Award.

[5] Harel, D. Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*. North-Holland, 1987, vol. 8, no. 3, p. 231–274. ISSN 0167-6423.

[6] Hopcroft, J. E.; Motwani, R. and Ullman, J. D. *Introduction to Automata Theory, Languages, and Computation*. 3rd ed. Upper Saddle River, NJ: Pearson Education, 2006. ISBN 978-0-321-45536-9.

[7] Ierusalimschy, R.; de Figueiredo, L. H. and Celes, W. *Lua 5.4 Reference Manual* online. Lua.org, PUC-Rio, 2020. Available at: https://www.lua.org/manual/5.4/. [cit. 2025-05-11].

[8] Ierusalimschy, R.; de Figueiredo, L. H. and Celes Filho, W. Lua—An Extensible Extension Language. *Software: Practice and Experience*. Wiley, 1996, vol. 26, no. 6, p. 635–652. ISSN 1097-024X. Available at: https://doi.org/10.1002/(SICI)1097-024X(199606)26:6<635::AID-SPE26>3.0.CO;2-P.

[9] Machine Zone, Inc.. *IXWebSocket: WebSocket and HTTP Client and Server Library* online. 2024. Available at: https://github.com/machinezone/IXWebSocket. [cit. 2025-05-11]. BSD 3-Clause License.

[10] Meta Open Source. *React: The Library for Web and Native User Interfaces* online. 2024. Available at: https://react.dev/. [cit. 2025-05-11].

[11] Murata, T. Petri Nets: Properties, Analysis and Applications. *Proceedings of the IEEE*. IEEE, 1989, vol. 77, no. 4, p. 541–580. ISSN 0018-9219.

[12] Noaman, M.; Khan, M. S.; Abrar, M. F.; Ali, S.; Alvi, A. et al. Challenges in Integration of Heterogeneous Internet of Things. *Scientific Programming*. Hindawi, 2022, vol. 2022, p. 1–17.

[13] O'Leary, N. and Conway-Jones, D. *Node-RED: Low-Code Programming for Event-Driven Applications* online. 2013. Available at: https://nodered.org/. [cit. 2025-05-11]. Open-sourced September 2013; now part of the OpenJS Foundation.

[14] OpenJS Foundation. *Electron: Build Cross-Platform Desktop Apps with JavaScript, HTML, and CSS* online. 2024. Available at: https://www.electronjs.org/. [cit. 2025-05-11].

[15] Petri, C. A. *Kommunikation mit Automaten*. Darmstadt, Germany, 1962. Dissertation. Technische Hochschule Darmstadt.

[16] Phoenix Framework Contributors. *Phoenix Framework — Peace of Mind from Prototype to Production* online. 2024. Available at: https://www.phoenixframework.org/. [cit. 2025-05-11].

[17] Poimandres (pmndrs). *Zustand: Bear Necessities for State Management in React* online. 2024. Available at: https://github.com/pmndrs/zustand. [cit. 2025-05-11]. MIT License.

[18] Strasser, T.; Rooker, M. N.; Ebenhofer, G.; Zoitl, A.; Sunder, C. et al. Framework for Distributed Industrial Automation and Control (4DIAC). In: *2008 6th IEEE International Conference on Industrial Informatics (INDIN)*. IEEE, 2008, p. 283–288.

[19] ThePhD. *Sol2: C++ ↔ Lua API Wrapper* online. 2023. Available at: https://github.com/ThePhD/sol2. [cit. 2025-05-11]. MIT License.

[20] Valim, J. et al. *Elixir Programming Language* online. 2024. Available at: https://elixir-lang.org/. [cit. 2025-05-11].

[21] YAML Language Development Team. *YAML Ain't Markup Language (YAML) Revision 1.2.2* online. 2021. Available at: https://yaml.org/spec/1.2.2/. [cit. 2025-05-11].

[22] Zoitl, A. and Strasser, T. *Distributed Control Applications: Guidelines, Design Patterns, and Application Examples with the IEC 61499*. Boca Raton, FL: CRC Press / Taylor & Francis, 2017. ISBN 978-1-482-25905-6.

Appendix A

Contents of the Included Storage Media

The included storage medium contains the following items:

`src/` Full source code of the Aetherium Automata platform:

- `src/engine/` — C++17 runtime engine and platform targets
- `src/ide/` — Electron IDE (TypeScript/React)
- `src/gateway/` — Elixir/Phoenix gateway
- `src/server/` — Elixir server

`example/` Showcase automata catalog:

- `example/automata/showcase/` — 18 YAML automata definitions
- `example/ide_demo_projects/` — IDE demo project files (`.aeth`)

`thesis/` LaTeX source of this technical report and compiled PDF.

Appendix B

YAML DSL Schema Reference

The complete field reference for the YAML domain-specific language is given below. Fields marked **(R)** are required; fields marked **(O)** are optional.

Table B.1: Top-level YAML fields.

Field	Required	Description
`version`	R	Schema version (currently `0.0.1`)
`config`	R	Automaton metadata block
`variables`	O	List of variable definitions
`automata`	R	EFSM body

Table B.2: `config` block fields.

Field	Required	Description
`name`	R	Human-readable automaton name
`type`	R	`inline` or `folder`
`location`	R if `folder`	Path to folder containing code files
`language`	O	Scripting language; default `lua`
`description`	O	Free-text description
`author`	O	Author name
`tags`	O	List of string tags for discovery

Table B.3: Transition type-specific fields.

Type	Required fields	Optional fields
`classic`	`condition`	`priority`, `weight`
`timed`	`timed.mode`, `timed.after` (mode-dependent)	`timed.jitter`, `priority`
`event`	`event.triggers`	`event.require_all`, `priority`
`probabilistic`	`weight`	`priority`
`immediate`	—	`priority`

Appendix C

Supported Platform Targets

Table C.1: Engine platform targets and their capability profiles.

Target	Architecture	Script engine	Transport	Build system
Desktop Linux/macOS	x86-64 / ARM64	Lua 5.4 (Sol2)	IXWebSocket	CMake
ESP32	Xtensa LX6	Lightweight / no-op	Serial / MQTT	Arduino/CMake
Raspberry Pi Pico	ARM Cortex-M0+	No-op stub	Serial	CMake
MCXN947	ARM Cortex-M7	Lightweight	Serial	CMake